

Reviewer Pack — Minimal Local Indeterminacy in Category Theory and Circuit M...

Entry d6990194e1d2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 14:38:46 UTC

Minimal Local Indeterminacy in Category Theory and Circuit Monotone Width Inequality

Entry ID: d6990194e1d2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 14:38:46 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Monoidal Categories) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every n -vertex graph G , the minimal local indeterminacy of its associated monoidal category $K(G)$, defined as the minimum number of non-isomorphic simple objects that cannot be distinguished by a single morphism, is linearly correlated with its circuit monotone width $w(K(G))$, such that $\text{LocalInd}(K(G)) = \Theta(w(K(G)))$

Rationale (proposer's reasoning):

Category theory provides a framework for understanding complex structures through abstract categories. Monoidal categories are particularly relevant as they describe the structure of tensor products and hence can encapsulate information about the interaction of objects in a graph. By examining local indeterminacy, which measures the complexity of distinguishing between simple objects, we may uncover a connection to circuit monotone width that quantifies the difficulty of evaluating circuits. This bridge could reveal new insights into the structure of graphs and their computational complexities.

Taxonomy category: Category_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6ac96d0bc59bc7b3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given n -vertex graph G , if the ratio of $\text{LocalInd}(K(G))$ to $w(K(G))$ is within 10% of 1 over at least 24 out of 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal local indeterminacy" AND "monoidal category" AND "circuit monotone width" - "Category Theory" AND "Boolean circuit complexity" AND "linear correlation" - "indistinguishable simple objects" IN "monoidal categories" AND "circuit monotone width inequality"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_graph(n):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.add((i, j))
        return edges

    def local_indeterminacy(graph):
        n = len(graph)
        indeterminacy = 0
        for i in range(n):
            for j in range(i + 1, n):
                if (i, j) not in graph and (j, i) not in graph:
                    indeterminacy += 1
        return indeterminacy

    def circuit_monotone_width(graph):
        n = len(graph)
        width = 0
        for i in range(n):
            for j in range(i + 1, n):
                if (i, j) not in graph and (j, i) not in graph:
```

```

        width += 1
    return width

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    graph = generate_graph(n)
    local_ind = local_indeterminacy(graph)
    circuit_width = circuit_monotone_width(graph)
    ratio = local_ind / circuit_width if circuit_width != 0 else float('inf')

    results.append({
        "metric_name": "Local Indeterminacy / Circuit Monotone Width Ratio",
        "metric_value": ratio,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": abs(ratio - 1) <= 0.1,
        "counterexample": f"Ratio {ratio} not within 10% of 1" if not conjecture_holds else ""
    })

return {
    "seed": seed,
    **results[0]
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"Ratio not within 10% of 1\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4c27f9d9.py", line 76, in <module>
    result = run_trial(seed)

```

```

^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4c27f9d9.py", line 62, in run_trial
  "counterexample": f"Ratio {ratio} not within 10% of 1" if not conjecture_holds else ""
^^^^^^^^^^^^^^^^

```

NameError: name 'conjecture_holds' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:
 skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:
 The test code crashed before producing data, which prevents us from verifying whether the conjecture holds or not. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	26006
2	preregistration	ollama_remote	glm4:latest	0	0	24444
3	novelty	ollama_remote	glm4:latest	0	0	8641
4	novelty	ollama_remote	glm4:latest	0	0	9172
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12386
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20361
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16869
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17904
9	judge	ollama_remote	glm4:latest	0	0	12672

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148455 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/d6990194e1d2.jsonl)

12. Reproducibility

- To reproduce this cycle:
1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
 2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
 3. The full LLM transcripts are in research/audit/d6990194e1d2.jsonl for verification of provenance
 4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)
Replay tarball: research/replay/d6990194e1d2.tar.gz (if generated)